LMJuly18-05 — LaunchMind: Intelligence Systems & Agent Platform

Date: July 18, 2026 · Part 5 of 6

Table of Contents

1. AI Platform Architecture

Overview

All AI calls in LaunchMind flow through a single mandatory pipeline:

```
Service code
  → callSonnet() or callHaiku() in aiPlatform.ts
    → sanitizeInput() — prompt injection defense
    → routeModel() — select correct model + maxTokens
    → aiClient.ts — raw Anthropic SDK call
      → retries (2×, 500ms→1000ms backoff)
      → timeout (60s Sonnet / 30s Haiku)
    → stripMarkdownFences() — clean model output
    → Zod validation (if schema provided)
    → write to ai_requests table (success or failed)
    → return parsed result
```

`aiPlatform.ts` Public API

```
// Required AuditContext — both fields are optional per-field but the object is required
type AuditContext = {
  founderId?:  string | null;
  productId?:  string | null;
  promptId:    string;   // required
  action:      string;   // required
}


// Sonnet: complex generation (strategy, briefs, content, reports)


export async function callSonnet(
  system:    string,
  user:      string,
  maxTokens: number,
  auditCtx:  AuditContext,
  schema?:   z.ZodSchema   // optional — forces JSON output + validates
): Promise<string>


// Haiku: fast classification, scoring, short copy


export async function callHaiku(
  prompt:    string,
  maxTokens: number,
  auditCtx:  AuditContext
): Promise<string>


// Full pipeline with context + prompt registry


export async function generateAI(opts: {
  founderId:  string;
  productId:  string;
  promptName: string;
  userInput:  string;
  maxTokens?: number;
}): Promise<AIResponse>
```

Error Handling

Error type	Behaviour
429 (rate limit)	Retry with 500ms / 1000ms backoff (max 2 retries)
500/503 (server error)	Retry same backoff
Timeout (60s/30s exceeded)	Write `status='timeout'` to ai_requests, throw
Zod validation failure	Throw `OutputValidationError`, write `status='failed', error_message='output_validation_failed'` to ai_requests
Invalid API key	Throw immediately, write `status='failed'`

AI Cost Table

```
const COST_TABLE = {
  'claude-sonnet-4-6': {
    inputPerMToken:  3.00,   // USD
    outputPerMToken: 15.00,  // USD
  },
  'claude-haiku-4-5-20251001': {
    inputPerMToken:  0.25,   // USD
    outputPerMToken: 1.25,   // USD
  },
}
```

Costs are computed from `usage.input_tokens` + `usage.output_tokens` and stored in `ai_requests.cost_usd`.

Prompt Injection Defense

`sanitizeInput(text: string): string` strips:

- Role markers: `system:`, `user:`, `assistant:`, `<system>`, etc.
- Instruction overrides: `ignore previous`, `new instruction`, `disregard`, `forget`, etc.
- Injection framing: `[INST]`, `<<SYS>>`, etc.

Applied to all user-provided text before it enters any prompt.

---

2. Context Engine

`lib/contextEngine.ts` — builds rich context packages for AI calls.

```
export interface ContextPackage {
  product?:    { id, name, confirmedIcp, markets, platform, brandVoiceProfile };
  campaigns?:  Array<{ id, channel, market, status, metrics? }>;
  memories?:   Array<{ id, title, body, memoryType, confidence }>;
  graph?:      { nodes: KnowledgeNode[]; edges: KnowledgeEdge[] };
  analytics?:  { totalInstalls, avgCpi, topChannel, weekOverWeekDelta };
  recentEvents?: LearningEvent[];
}

export async function buildContextPackage(

  founderId: string,
  productId: string,
  opts?: {
    includeMemories?:      boolean;  // default false
    includeKnowledgeGraph?: boolean; // default false
    maxMemories?:          number;   // default 3
  }
): Promise<ContextPackage>
```

Non-fatal per source: each of the 6 parallel queries is wrapped in try/catch. One source failing (e.g., no analytics data) doesn't fail the whole context build. Missing fields are `undefined` and prompts must handle gracefully. **6 parallel data sources**:

- `products` table — confirmed_icp, brand_voice_profile, markets, platform
- `campaigns` + `campaign_metrics` — active campaigns + recent metrics
- `marketing_memories` — top-N by confidence (if `includeMemories: true`)
- `knowledge_nodes` + `knowledge_edges` — depth-1 graph traversal (if `includeKnowledgeGraph: true`)
- `campaign_metrics` aggregation — totalInstalls, avgCpi, topChannel
- `learning_events` — 5 most recent, processed=true

```
export function formatContextForPrompt(pkg: ContextPackage): string
// Returns human-readable string like:
// "Product: ClientPulse (App Store). Markets: india. ICP: {...}.
//  Active campaigns: 2. Top channel: whatsapp. Total installs: 847."
```

---

3. Prompt Registry

`lib/promptRegistry.ts` — versioned prompt management.

```
export async function resolvePrompt(name: string): Promise<Prompt | null>
// Returns the active version of a named prompt from the prompts table

export async function registerPrompt(

  name: string, body: string, model: string, maxTokens: number
): Promise<Prompt>
// Auto-increments version number
// Archives previous active version (active=false) before inserting new

export async function listPrompts(): Promise<Prompt[]>

// Returns all active prompts (latest version per name)

export async function listPromptVersions(name: string): Promise<Prompt[]>

// All versions of a prompt, ordered by version desc
```

11 Seeded Prompts (migration 043)

Prompt name	Model	Usage
morning_brief	sonnet	Morning Brief AI recommendation
strategy_generation	sonnet	30/60/90 day strategy
content_assets_generation	sonnet	Full content batch (all asset types)
brand_voice_extract	haiku	Extract brand voice profile from product copy
brand_voice_apply	haiku	Rewrite text in brand voice
icp_structuring	haiku	Structure scraped data into ICP
review_analysis	haiku	Sentiment + theme analysis of reviews
weekly_brief	haiku	Weekly performance brief
content_score	haiku	Score content asset quality
recommendation_generation	haiku	Generate growth recommendations
agent_campaign_draft	haiku	Draft campaign configurations

Studio-Only Prompt Registration

`POST /ai/prompts` is gated behind `checkPlanFeature(plan, 'custom_prompts')`. Only Studio plan founders can register custom prompts via API.

---

4. Model Router

`lib/modelRouter.ts` — maps action names to model + maxTokens.

```
const ROUTING_TABLE: Record<string, { model: 'sonnet' | 'haiku'; maxTokens: number }> = {
  // Sonnet — complex, multi-step generation
  strategy_generation:       { model: 'sonnet', maxTokens: 8192  },
  morning_brief:             { model: 'sonnet', maxTokens: 512   },
  content_assets_generation: { model: 'sonnet', maxTokens: 12000 },
  // Haiku — fast, cheap, classification
  review_analysis:           { model: 'haiku',  maxTokens: 1024  },
  icp_structuring:           { model: 'haiku',  maxTokens: 2048  },
  brand_voice_extract:       { model: 'haiku',  maxTokens: 400   },
  brand_voice_apply:         { model: 'haiku',  maxTokens: 300   },
  content_score:             { model: 'haiku',  maxTokens: 512   },
  recommendation_generation: { model: 'haiku',  maxTokens: 2048  },
  agent_campaign_draft:      { model: 'haiku',  maxTokens: 1024  },
  weekly_brief:              { model: 'haiku',  maxTokens: 2048  },
  // Default fallback
  default:                   { model: 'haiku',  maxTokens: 1024  },
}

export function routeModel(promptId: string, maxOverride?: number): ModelConfig

// Returns { model: string; maxTokens: number }
// maxOverride: cap maxTokens lower (never higher) than table value

export function isSonnet(promptId: string): boolean
```

5. Image Generation Pipeline

`lib/replicateClient.ts`

Three image styles:

```
export type ImageStyle = 'photorealistic' 'graphic' 'mockup'


export async function generateImage(

  brief:   string,           // marketing context from brief
  style:   ImageStyle,
  options?: {
    logoUrl?:      string;   // URL to overlay logo
    brandColors?:  string[];
    screenshots?:  string[]; // existing app screenshots for context
  }
): Promise<string>           // Supabase Storage URL (permanent, not Replicate CDN URL)
```

Decision tree:

```
style=mockup  + marketingImages[] present → use real screenshot + logo composite
                                            model_used = 'real-screenshot+mockup+logo'
style=mockup  + no marketingImages        → Flux.1 photorealistic fallback
style=photo   + marketingImages present   → Flux.1 with enriched context from screenshots
style=photo   + no marketingImages        → Flux.1 photorealistic
style=graphic + any                       → Flux.1 graphic/illustration
```

Anti-split constants (prevent common Flux.1 failure modes):

```
const ANTI_SPLIT = 'no split screen, no diptych, no panels, no before/after';
const ANTI_TEXT  = 'no text, no words, no logos, no watermarks, no typography';
const ANTI_DARK  = 'bright natural lighting, warm tones, no cinematic dark shadows';
```

Emotion→lighting map: maps marketing emotions (excitement, trust, calm, joy) to specific lighting descriptors to ensure warm/bright outputs. `_extractPositiveScene(brief)`: strips "left shows X, right shows Y" split-screen patterns that cause Flux.1 to generate split compositions. **Logo compositing** (`_compositeLogoOntoImage`): applied after real screenshot OR Flux.1 generation. Places logo in top-right corner at 15% width, with 8px padding.

`services/marketingImagesService.ts`

At intake: called at 75% progress in `intakeWorker.ts`.

3 best-effort sources (individual failures don't fail the job):

- **App Store / Play Store screenshots**: from scraped_meta, download up to 5, upload to Supabase Storage
- **Website hero/feature images**: from website_meta.heroImages, download + upload
- **Google Custom Search**: `"${appName} mobile app"` query → download up to 3 results + upload

Storage path: `content-assets/{founderId}/{productId}/source-images/`

Returns: permanent Supabase Storage public URLs → stored in `products.scraped_meta.marketingImages[]`

Why permanent URLs: App Store CDN URLs expire after ~24h. Google image URLs can go down. Only Supabase Storage URLs are permanent and owned by LaunchMind.

6. Marketing Memory System

Memory Types

```
brand       — Brand voice, tone, visual identity observations
product     — Product positioning, features, differentiators
customer    — ICP insights, pain points, motivations
campaign    — Campaign performance observations
founder     — Founder preferences, decisions, context
channel     — Channel-specific learnings
market      — Market dynamics (USA / India)
competitor  — Competitor observations
experiment  — A/B experiment results
```

Memory Lifecycle

```
create → active (confidence auto-set, embedding generated async)
      → update (creates version record, increments version number)
      → archive (soft-delete, archived=true)
      → merge (source archived, target updated with merged content)
```

Deduplication Strategy

- **Exact-match dedup**: `findDuplicateMemory(founderId, title, type)` — synchronous, checked on every create
- **Vector similarity dedup**: async, runs after embedding generation — never auto-merges, flags for human review
- **Manual merge**: `POST /memory/merge` — founder or system explicitly merges two memories

Confidence Scoring

- Initial: derived from source quality (intake_completed = 0.7, campaign_result = 0.6, review_ingested = 0.55)
- Increases: `addEvidence()` adds supporting evidence; confidence adjusted upward
- Decreases: contradicting learning event lowers confidence
- Displayed: 0–100 scale in UI (multiply raw 0.0–1.0 × 100)

Seed Data (vijay@lm.com)

5 memories created from intake of ClientPulse:

title	memory_type	confidence
brand memory: clientpulse	brand	0.81
product memory: clientpulse	product	0.79

customer memory: clientpulse	customer	0.86
campaign memory: clientpulse	campaign	0.58
founder memory: clientpulse	founder	0.63

---

7. Knowledge Graph

Node Types

```
product, channel, market, competitor, audience_segment,
campaign_type, metric, insight, founder
```

Key Behaviours

- **Upsert-safe**: `createNode` with `UNIQUE(founder_id, node_type, label)` — safe to call multiple times
- **Owner verification**: `createEdge` verifies both source and target nodes are owned by the same `founderId` before creating the edge
- **Depth-1 traversal**: `getGraph()` returns all nodes + all edges between them (no multi-hop)
- **Plain English relations**: frontend renders edge `relation_type` as readable sentences ("Product X outperforms Channel Y")

Example Graph Nodes (from intake)

```
product:ClientPulse —[targets]—> audience_segment:SMB founders
product:ClientPulse —[performs_on]—> channel:WhatsApp
channel:WhatsApp    —[outperforms]—> channel:Meta_India
```

---

8. Learning Pipeline

`services/learningPipelineService.ts` — single entry point for all learning events.

```
export async function ingestLearningEvent(
  founderId:  string,
  eventType:  LearningEventType,
  payload:    Record<string, unknown>
): Promise<LearningResult>
```

8 Event Handlers

Event type	Trigger	Memory actions
`intake_completed`	Product intake finishes	Create product + brand + customer memories; create knowledge nodes for product + channels
`campaign_result`	Campaign metrics collected	Create/update campaign performance memory; update channel node confidence
`review_ingested`	New app reviews scraped	Create customer insight memory; update product node
`founder_feedback`	Founder rates a report	Create/update founder preference memory
`growth_brain_approved`	Founder approves strategy	Increase product memory confidence; create strategy memory
`analytics_synced`	Weekly analytics run	Create channel performance signal; trigger `generateRecommendations()` if insights high-confidence
`experiment_result`	Winner selected on experiment	Create experiment memory with learning_summary; update channel nodes
`ai_conversation`	Ask LaunchMind interaction	Create interaction memory (if insight-worthy)

Async vs Sync

- `intake_completed`, `campaign_result`, `review_ingested`, `founder_feedback`, `growth_brain_approved`: **synchronous** (direct function call)
- `analytics_synced`, `experiment_result`: **async via BullMQ** (heavy processing, don't block the request)

---

9. Agent Platform & Mission Orchestrator

Mission Lifecycle

```
draft → queued → running → [requires_approval] → completed
                                                → failed → (retry) → queued
                                                → cancelled
```

Approval at `requires_approval`: founder approves → step completes + re-queues. Founder rejects → mission cancelled.

Agent Types (12)

Agent	Status	What it does
`research`	Full	Scrapes product category + competitors; returns structured research package
`strategy`	Full	Generates 30/60/90 day strategy via Sonnet; calls `strategyService.generateStrategy()`
`content`	Full	Generates content asset batch; calls `contentService.generateContentAssets()`
`campaign`	Full	Drafts campaign configs; validates each draft against §1.6 spend cap per channel
`memory`	Full	Creates/updates memories based on input; calls `marketingMemoryService`
`reporting`	Full	Generates weekly/monthly report; calls `reportingService.generateReport()`
`planning`	Stub	Returns placeholder — M13 implementation pending
`creative`	Stub	Returns placeholder — M13 implementation pending
`publishing`	Stub	Returns placeholder; enforces §1.5 (checks `approved_at` before any publish action)
`optimization`	Stub	Returns placeholder — M13 implementation pending
`learning`	Stub	Returns placeholder — M13 implementation pending
`benchmark`	Stub	Returns placeholder — M13 implementation pending

`AgentFn` Type

```
type AgentFn = async (
  input:   unknown,
  context: AgentContext
) => Promise<unknown>


interface AgentContext {


  contextPkg: ContextPackage;
  founderId:  string;
  productId:  string | null;
  log: (message: string, level: LogLevel, metadata?: Record<string, unknown>) => Promise<void>;
}
```

Mission Worker (`missionWorker.ts`)

- BullMQ queue: `mission-execution`
- Concurrency: 5 parallel missions
- Priority: from `MISSION_PRIORITY` (numeric, 1=highest)
- DLQ: no separate queue — failed missions set `missions.status='failed'` (queryable)

- Retry: explicit via `POST /missions/:id/retry` (founder-triggered, not auto-retry)

Idempotency

`createMission()` accepts an optional `idempotencyKey`. If a mission with the same key exists and is not `failed` / `cancelled`, returns the existing mission. Prevents duplicate missions from UI double-clicks.

Mission Step Dispatch

```
// In missionWorker:
const step = await missionService.getNextPendingStep(missionId);
const agentFn = AGENT_REGISTRY[step.agent_type];
const result  = await agentFn(step.input, context);
await missionService.completeStep(step.id, result);
```

---

10. Decision Engine

`services/decisionEngineService.ts` — 8 pure TypeScript rules. **Zero AI calls. AI cannot override.**

```
export class DecisionError extends Error {
  statusCode: number;  // HTTP status to return
  code:       string;  // machine-readable code
  detail:     string;  // human-readable explanation
}
```

8 Rules

```
// §1.5 — Approve-before-post
export function checkApprovalGate(campaign: { approved_at: string | null }): void
// throws DecisionError(422, 'APPROVAL_REQUIRED', 'Campaign must be approved before launching')


// §1.6 — Spend cap

export function checkSpendCap(
  campaign: { spend_cap: { weekly_usd?: number } | null },
  proposedBudget: number
): void
// throws DecisionError(422, 'SPEND_CAP_EXCEEDED', Proposed budget $${X} exceeds cap $${Y})


// Plan feature gate

export function checkPlanFeature(plan: string, feature: string): void
// throws DecisionError(403, 'PLAN_UPGRADE_REQUIRED', Feature requires ${minPlan} plan)


// Token balance

export function checkTokenBalance(balance: number | null, required: number): void
// throws DecisionError(402, 'INSUFFICIENT_TOKENS', Need ${required} tokens, have ${balance})


// Content regen limit (sync DB read)

export function checkRegenLimit(currentCount: number, limit: number): void
// throws DecisionError(429, 'REGEN_LIMIT_REACHED', Max ${limit} regenerations reached)


// Experiment runtime (min 24h before selecting winner)

export function checkExperimentRuntime(startDate: string, minHours?: number): void
// throws DecisionError(422, 'EXPERIMENT_TOO_SHORT', 'Minimum runtime not met')


// Workspace permission

export function checkWorkspacePermission(
  member: { role: 'owner' 'admin' 'member' },
  requiredRole: 'owner' | 'admin'
): void
// throws DecisionError(403, 'INSUFFICIENT_WORKSPACE_ROLE', ...)


// Benchmark access (no-op — all founders can access benchmarks)

export function checkBenchmarkAccess(founderId: string): void
// never throws
```

Plan Feature Map

Feature	Minimum plan
`playbook_insights`	builder
`india_market`	solo
`workspaces`	studio
`custom_prompts`	studio
`api_keys`	studio
`recommendations`	builder
`ai_audit`	solo

---

11. Intelligence Network & Benchmarks

`services/intelligenceNetworkService.ts`

```
export async function ingestCampaignOutcome(
  category: string, market: string, channel: string,
  metrics: { installDelta, conversionRate, retentionD7, hookType, priceTier }
): Promise<void>
// Privacy guard: only inserts to intelligence_trends if signal cohort ≥ 3
// No founder_id, no product_id in intelligence_trends (fully anonymous)

export async function getBenchmarks(

  category: string, market: string
): Promise<BenchmarkResult | null>
// Returns aggregates if signal_count >= 3, else null

export async function getTrends(

  category: string, market: string, period: '30d' | '90d'
): Promise<TrendSummary>

export async function computeTrends(): Promise<void>

// Called by weekly cron: aggregates recent signals into intelligence_trends
```

Anonymization Guarantee

`intelligence_trends` stores only:

- `category`, `market`, `channel` — categorical
- `period_start`, `period_end` — time range
- `signal_count` — cohort size (min 3 required)
- `avg_install_delta`, `avg_conversion`, `avg_retention_d7` — averages
- `top_hook_type` — most common hook type in cohort

No `founder_id`, no `product_id`, no `product_name`. The row itself cannot be used to identify any founder.

Benchmark Disclosure

When `signalCount < 20` on the Market Intelligence page, a yellow banner shows: > "These benchmarks are based on synthetic seed data. Benchmarks improve as more founders use LaunchMind."

---

12. Recommendation Engine

`services/recommendationEngineService.ts`

```
export async function generateRecommendations(
  founderId: string,
  productId: string
): Promise<Recommendation[]>
```

Generation flow:

- `buildContextPackage(founderId, productId)` — gather context
- `callHaiku(prompt, 2048, auditCtx)` — generate JSON array of recommendations
- Zod validate each recommendation
- Deduplicate by title (case-insensitive) against existing active recommendations
- Set `expires_at = now() + 14 days`
- Upsert to `saved_opportunities` with recommendation-specific fields

Scoring formula (stored in `saved_opportunities.score`):

```
score = (impact × 0.4) + (confidence × 0.3) + (urgency × 0.2) + (source × 0.1)
```

Where:

- `impact`: 0.0–1.0 estimated business impact
- `confidence`: 0.0–1.0 confidence in the recommendation

- `urgency`: 0.0–1.0 time sensitivity
- `source`: quality weight by source type (growth_brain=1.0, analytics=0.9, memory=0.8, benchmark=0.7)

Plan gate: `POST /recommendations/generate` requires Builder+ plan (`checkPlanFeature(plan, 'recommendations')`). **Tenant isolation**: every recommendation endpoint adds `.eq('founder_id', founderId)` . FOUNDER_B cannot access FOUNDER_A's recommendations — verified in test suite.

---

13. Analytics & Optimization Engine

`services/analyticsService.ts`

```
// Cross-product totals for the founder
export async function getAnalyticsSummary(founderId: string): Promise<{
  totalInstalls: number;
  avgCpi: number | null;
  activeCampaigns: number;
  completedMissions: number;
  productBreakdown: Array<{ productId, name, installs, cpi }>;
}>


// Weekly install series for trend chart


export async function getKPITrend(
  founderId:  string,
  productId?: string,
  weeks?:     number   // default 12
): Promise<Array<{ weekStart: string; installs: number; cpi: number | null }>>


// Last-touch attribution by channel


export async function getAttribution(founderId: string): Promise<{
  byChannel: Array<{ channel, installs, percentage }>;
}>
// Last-touch: credit installs to the channel in campaign_metrics (no separate attribution table)


// Conversion funnel


export async function getFunnel(founderId: string): Promise<{
  impressions: number;
  clicks: number;
  installs: number;
  impressionsToClicks: number;   // %
  clicksToInstalls: number;      // %
}>


// ROI by channel


export async function getROI(founderId: string): Promise<{
  byChannel: Array<{ channel, spend, revenue, roi }>;
  totals: { spend, revenue, roi };
}>
// spend proxy  = CPI × installs
// revenue proxy = ROAS × spend
```

services/optimizationEngineService.ts

```
export async function generateInsights(
  founderId:  string,
  productId?: string
): Promise<{ created: number }>
// callHaiku → 3 insights per product → dedup by (founder, product, type) before insert
// High-confidence insights (≥0.8) automatically:
//   → trigger generateRecommendations()
//   → trigger ingestLearningEvent('analytics_synced')


export async function listInsights(


  founderId:  string,
  productId?: string
): Promise<OptimizationInsight[]>


export async function updateInsightStatus(


  insightId: string,
  status:    'applied' | 'dismissed'
): Promise<void>
```

6 Insight Types:

```
channel_shift         — Suggests reallocating budget to better-performing channel
budget_reallocation   — Spend cap adjustment recommendation
copy_refresh          — Content asset showing fatigue (CTR decline)
audience_expand       — New audience segment opportunity
timing_optimization   — Best day/time to post based on engagement data
market_opportunity    — Expand to new market (usually USA→India or vice versa)
```

services/reportingService.ts

Cache strategy: check `reports` table for `UNIQUE(founder_id, product_id, type, period_start)` before calling AI. Return cached if found.

Report type	Model	Trigger
`weekly`	Haiku	Manual or Sunday cron
`monthly`	Sonnet	Manual or month-end
`executive`	Sonnet	Manual
`campaign`	Haiku	After campaign completes
`experiment`	Haiku	After experiment winner selected

Post weekly report: `ingestLearningEvent('founder_feedback', { reportId, type: 'weekly' })` — creates a learning event that may generate memories from the report content.

Report content shape (`reports.content` JSONB):

```
{
  headline:    string;    // 1-sentence summary
  whatWorked:  string;    // top 2-3 wins
  fix:         string;    // top priority fix
  insights:    string[];  // 3-5 AI insights
  actions:     string[];  // concrete next steps
  generatedAt: string;    // ISO timestamp
}
```

---

14. Morning Brief AI System

`BRIEF_SYSTEM` Prompt (in `owner.route.ts`)

```
You are LaunchMind, an AI CMO analyzing app marketing performance.

Generate ONE clear, actionable recommendation for today's morning brief.



CRITICAL: Return ONLY a raw JSON object. No preamble, no explanation, no markdown code fences.
The response must be valid JSON that can be passed directly to JSON.parse().

Return exactly this shape:

{
  "title":       "Short action-oriented headline (max 12 words)",
  "summary":     "2-3 sentence explanation of what to do and why",
  "whyNow":      "Why this is the right move today specifically",
  "confidence":  <number 0-100>,
  "evidence":    ["supporting data point 1", "supporting data point 2"],
  "action":      "CTA button text (max 5 words)",
  "missionType": "<mission type string or null>"
}
```

`RecommendationSchema` (Zod validation)

```
const RecommendationSchema = z.object({
  title:       z.string(),
  summary:     z.string(),
  whyNow:      z.string(),
  confidence:  z.number().min(0).max(100),
  evidence:    z.array(z.string()),
  action:      z.string(),
  missionType: z.string().nullable(),
});
```

If model returns non-JSON output, aiPlatform.ts catches the parse failure, writes `status='failed'` to `ai_requests`, and `recommendation` is set to `null`. The frontend shows `RecommendationUnavailable` with a "Try again" button.

Morning Brief Response Assembly

The `/owner/brief` endpoint runs 7 parallel queries + 1 AI call:

- `founders` — name, plan, token_balance
- `getActiveProduct()` — most recent product with intake_step=6
- `getPendingApprovals()` — missions + campaigns requiring approval
- `saved_opportunities` — top-3 active/saved by confidence
- `mission_logs` — recent info/warn logs (timeline)
- `campaign_metrics` — last 10 weeks for delta + CPI computation
- `marketing_memories` — top-3 by confidence ← **Added in July 18 session**
- `callSonnet(BRIEF_SYSTEM, contextSummary, 512, auditCtx, RecommendationSchema)` — AI recommendation

Week-over-week install delta computation:

```
const weekBuckets = Object.entries(
  allMetrics.reduce((acc, m) => {
    const w = m.week_start;
    acc[w] = (acc[w] ?? 0) + (m.installs ?? 0);
    return acc;
  }, {})
).sort(([a], [b]) => b.localeCompare(a)); // descending (most recent first)


// null if < 2 weeks of data or lastWeek === 0

const weekOverWeekInstallDelta =
  weekBuckets.length >= 2 && weekBuckets[1][1] > 0
    ? Number(((weekBuckets[0][1] - weekBuckets[1][1]) / weekBuckets[1][1] * 100).toFixed(1))
    : null;
```

Continue to: LMJuly18-06-Build-State-Roadmap.md